

Little background on Homomorphic Encryption

Homomorphic Encryption is cryptographic scheme which allow computations to be performed **directly on encrypted data**, producing an encrypted result that, when decrypted, is identical to what you would have obtained by performing the same computation on the unencrypted data.

To make this concrete in our context: a vehicle encrypts its model parameters before sending them. The aggregation server receives encrypted parameters, ciphertexts and performs the computation directly on these ciphertexts, without ever decrypting them. **The result of this operation is itself a ciphertext. The server sends this ciphertext back to the vehicles.** Each vehicle decrypts it using its own private key and obtains the updated global model parameters. **The server, at no point in this entire process**, ever sees any plaintext model parameters. It performs its computation in the blind.

Now, there are various HE schemes:

Not all Homomorphic Encryption schemes are the same. They differ in:

- what types of computations they support,
- how efficiently they support those computations,
- and what data types they work with natively.

I have provided the list of HE schemes in the document shared earlier. (For future reference). Nevertheless, the current scheme for this project (as I found out) is the CKKS scheme.

There is a more advanced hybrid framework called **PEGASUS** which can be useful. You can do a further study on it to see if it matches our project requirements but I think it would be an overkill.

The issue (FHE is not practical in our case)

The name of our project says "**Fully Homomorphic Encryption.**" There is an important distinction to understand, because the word "fully" carries a very specific technical meaning that has direct consequences for what we can actually implement.

Homomorphic Encryption exists in a spectrum.

1. Partially Homomorphic Encryption supports **either addition or multiplication but not both.**
2. Somewhat Homomorphic Encryption (SHE) or Levelled Homomorphic Encryption (LHE) **supports both addition and multiplication, but only up to a limited depth** i.e. a limited number of sequential multiplication operations before the noise becomes too large.
3. Fully Homomorphic Encryption (FHE) **supports both addition and multiplication an unlimited number of times.**

The mechanism that enables unlimited computation in FHE is called **bootstrapping**. (Just in case you the team asks you why FHE is not supported, just tell them this reason that it is because of bootstrapping)

The refined conclusion we can state as: **"Full FHE with bootstrapping"** is not feasible on the hardware where this system must run.

Concept of Bootstrapping

Every FHE operation injects a small amount of mathematical noise into the ciphertext to protect it from strangers.

But every cipher text has a limit to how much noise it can handle in itself. That limit is called as **noise budget**. (I will use this term from now on, whenever you encounter this term, just know that it's just a threshold) If the cumulative noise from all operations exceeds this threshold, then the decryption of the cipher text produces a garbage value instead of the correct result.

Remember the following:

- 1) Addition introduces very less noise to the CT (Cipher text). Means, addition does not consume the noise budget that much.
- 2) Multiplication is catastrophically expensive and eats up the noise budget very quickly.

In our project, I chose a method called **levelled HE**.

Bootstrapping is nothing but refreshing this noise budget. It's like filling the container again with material so that it can be used again. Its just recharge. Hence, bootstrapping a way to **refresh** a ciphertext by reducing its noise back to a safe level without actually decrypting it (which would expose the secret data).

Why bootstrapping is an issue for our project

The problem is the hardware cost and time. The time needed for bootstrapping (depending upon the parameters) will be very huge. In our case, we have 45998 parameters, the time for bootstrapping would be very large.

Also, there would be an issue of memory because bootstrapping also makes use of keys (known as **bootstrapping keys**) that should be stored in the memory. A typical automotive ECU has 2 to 8 GB of RAM in high-end configurations and as little as 256 MB to 512 MB in standard configurations.

So, we need to follow a different approach: **Levelled Homomorphic Encryption** or **Somewhat Homomorphic Encryption**, specifically using CKKS **without bootstrapping**. (Already mentioned in the prior document). This approach will support addition and a limited number of multiplications just enough for FL aggregation.

The first phase of work on the FHE track has been entirely theoretical and analytical, which is both expected and necessary.